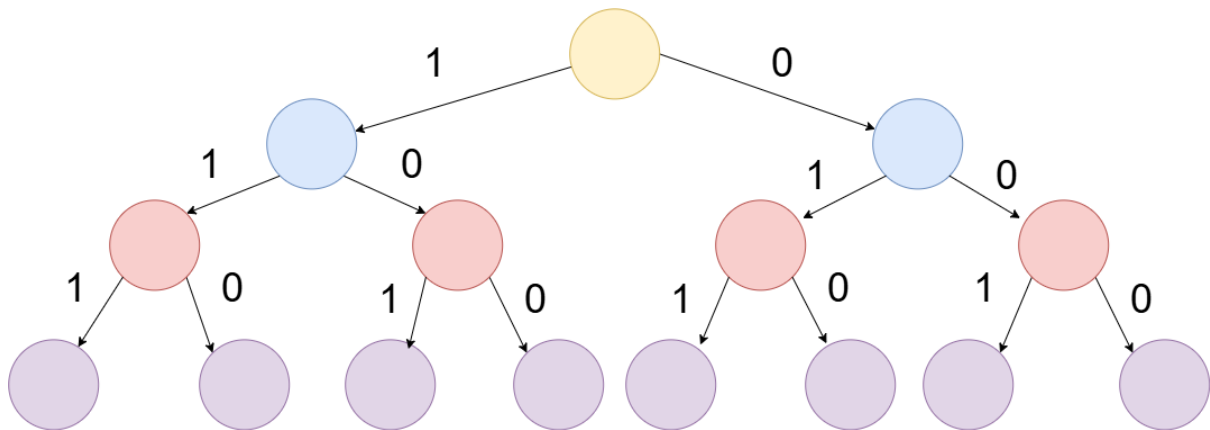


El ladrón audaz.

Paula María Arroyo Botica.

1. Descripción de la lógica del programa.

Para resolver el problema se ha usado el algoritmo de vuelta atrás o back tracking, que se basa en la exploración de un árbol de búsqueda, donde podemos analizar todas las soluciones posibles. En este caso, cada nivel del árbol (diferenciados con colores) representa uno de los objetos disponibles junto con las decisiones asociadas a él: Robar el objeto (1) o no robarlo (0).



A medida que se van tomando las diferentes decisiones sobre los objetos se obtiene una solución parcial y su beneficio asociado. Cuando se hayan analizado todas las opciones obtendremos una solución factible, que comparará con la mejor solución obtenida hasta el momento.

Si el beneficio de esta solución es mayor al de la mejor solución almacenada, la lista se actualiza. El proceso seguirá hasta haber recorrido todas las ramas del árbol, asegurándonos así de obtener la solución óptima.

2. Descripción de la implementación.

El programa se ha resuelto en java usando tres clases: Objeto, Ladron y ElLadronAudaz.

- **Objeto:** Esta clase se encarga de representar a los objetos, almacenando los atributos de “valor” y “peso”.
- **Ladron:** Incluye la función del algoritmo de vuelta atrás, que resuelve el problema mediante un método recursivo, que nos permite explorar todas las decisiones tomando en cuenta las tomadas anteriormente. Además, contiene las condiciones de decisión y de parada y las restricciones para resolver el problema.

- **ELLadronAudaz:** Esta es la clase principal, encargada de leer los ficheros de leer los ficheros de ejemplo y ejecutar el programa.

3. Discusión sobre si el programa proporciona o no la asignación óptima.

En este caso el programa nos proporciona la solución óptima. Esto es debido a que la vuelta atrás explora todas las posibles opciones quedándose solo con la óptima (la que tiene un mayor valor).

En cada solución factible optima se calcula su valor total y se compara con el valor calculado de la mejor solución obtenida hasta el momento. Si el nuevo beneficio es mayor que el anterior, la solución se actualiza. Así nos aseguraremos de que al final de la ejecución la solución almacenada sea la óptima.

4. Una evaluación de las limitaciones del programa.

La limitación que tiene el problema es su coste computacional. El algoritmo de vuelta atrás examina todas las posibles opciones de cada objeto, lo que provoca que el número de combinaciones aumente a medida que aumenta el tamaño del problema.

Si tenemos n elementos a elegir, como tenemos que examinar dos decisiones (robarlo o no robarlo) por paso, se harían en total 2^n operaciones. El crecimiento exponencial hace que el tiempo de ejecución del programa sea muy pequeño para casos en los que tengamos muy pocos objetos, pero que aumente considerablemente para casos con más objetos.

Otro algoritmo que se podría usar es el algoritmo voraz, que selecciona la mejor opción por cada paso. Sin embargo, pese a que este algoritmo es mucho más eficiente porque se hacen menos cálculos, no nos garantiza alcanzar la solución óptima en algunos ejemplos.